东莞理工学院计算机科学与技术学院

实验报告模板

课程名称：Qt 应用程序开发 **学期：2026 年秋季**

实验名称	带键盘事件的计算器			实验序号	一
姓　　名	王俊康	学　　号		班　　级	
实验地点		实验日期	2026 年 10 月 8 日	指导老师	
教师评语				实验成绩	

一、实验目标：

1) 熟悉 Qt Creator 开发环境的配置，理解 GUI 应用程序的基本框架，能熟练使用开发工具进行代码的编写、编译、调试和运行；

2) 掌握 UI 设计器的使用，能熟练设置各类 UI 组件的属性；

3) 掌握 Qt UI 界面布局的原理，能利用各种布局合理安排控件的位置；

4) 理解 Qt 信号与槽的原理，掌握信号与槽关联函数的使用和槽函数的定义；

5) 掌握键盘事件、输入状态管理和异常输入处理方法，能够分析 GUI 程序的交互逻辑；

6) 能够合理使用生成式 AI 或 AI Coding 工具辅助程序设计，并对生成代码进行验证、调试、修改和完善。

二、实验条件：

系统要求：Windows 64 位 或者 Mac OS 或者 Linux 均可

软件条件：Qt 6.9.2，Git

参考资料：

1) https://www.compart.com/en/unicode/

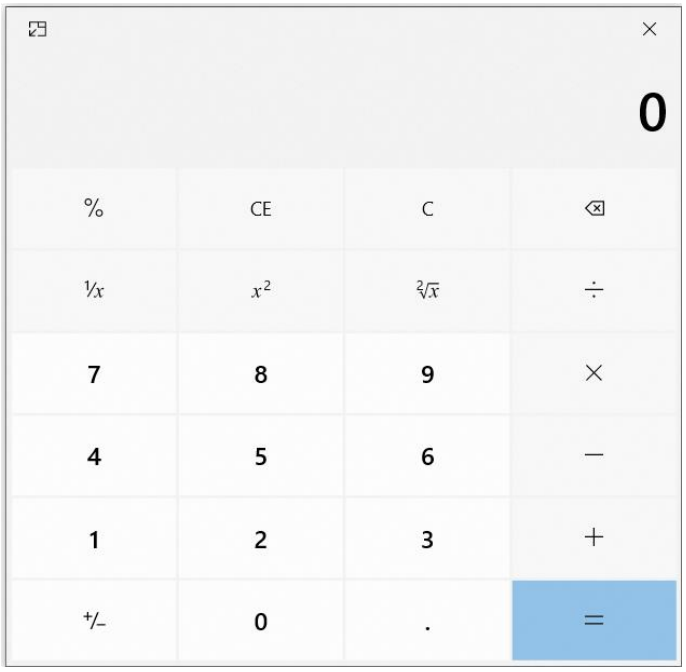
2) https://htmlcolorcodes.com/

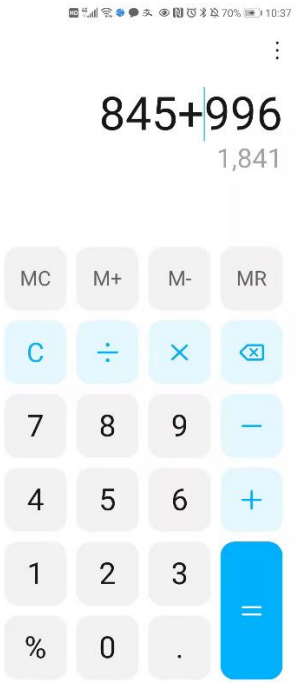
3) https://doc.qt.io/qt-5/qtwidgets-widgets-calculator-example.html

三、实验内容：

设计一个带键盘事件的计算器，可以实现加减乘除运算。本实验允许使用生成式 AI 或 AI Coding 工具辅助开发，但学生应对最终程序的正确性、完整性和代码质量负责，具体要求如下：

1) 界面设计可参考手机计算器或者 Windows 计算器，如下图，要求利用 UI 设计器完成界面组件属性的设置和界面的布局





2) 计算器基本功能同常用计算器，能完成四则运算、小数输入、退格以及清除等功能

3) 要求按键输入时，在窗口能正确显示输入的数据、操作符以及结果，并正确处理连续小数点、连续操作符等异常输入

4) 能用键盘代替按键完成计算功能，鼠标输入与键盘输入应保持一致，并尽可能复用同一套处理逻辑

5) 需要使用 GitHub 将代码管理起来，有体现代码迭代、测试和问题修复的过程

6) 程序应正确处理除数为 0、计算完成后继续输入、连续运算、输入过程中退格、清除后重新计算、鼠标与键盘混合输入等边界情况

7) 变量、函数和控件命名应清晰，避免大段重复代码，对关键状态和特殊处理添加必要注释

四、 实验步骤及结果：

1) UI 界面布局与设计：说明主要控件和布局方式

主要代码、设计说明及运行结果截图

2) 按键事件处理：完成操作数输入、小数点、退格和清除，并验证异常输入处理

说明如何避免同一操作数重复输入小数点，并给出关键代码及测试结果

3) 双操作数处理：完成四则运算及计算状态管理

说明程序如何管理第一个操作数、操作符、第二个操作数和计算结果等状态

4) 连续操作与边界处理：完成连续运算、除零及计算结果后的继续输入

至少选择 3 个异常或边界输入，记录：测试输入→原程序现象→原因分析→修改方法→修改后结果

5) 使用样式表改善界面，并检查控件命名与代码结构

给出优化后的界面截图，并说明主要改进

6) 键盘事件处理：实现键盘输入，并与鼠标按键处理逻辑保持一致

说明鼠标与键盘输入是否复用同一套处理逻辑，并给出关键代码及测试结果

五、Github 日志：

提交 GitHub 上与实验 1 相关日志，原则上不少于 5 次有效提交，应能体现功能实现、调试、问题修复和完善过程

六、AI 辅助开发记录：

选择 1 个具有代表性的 AI 辅助开发过程进行记录：问题→提示词→AI 给出的方案→实际运行结果→存在的问题→自己的修改。不要求完整粘贴全部 AI 对话。

七、测试结果：

录制 2～3 分钟测试视频，以附件形式提交到作业系统。视频应连续演示：鼠标完成带小数运算、键盘计算、退格和清除、除零处理、连续运算、至少 1 个异常输入处理，并展示 Git 提交历史

八、实验总结：

出错分析及解决过程，本次实验总结；结合 AI 辅助开发过程，说明 AI 提供了哪些帮助、存在什么问题，以及自己如何验证和改进 AI 生成的代码

实验报告正文

本次实验用 Qt 6 与 Qt Widgets 完成了一个带键盘事件的计算器：界面在 Qt 设计器中搭建，用垂直布局与网格布局安排显示屏和 19 个按键；运算部分用第一个操作数、待处理运算符和

“是否等待新操作数”三个状态量管理，鼠标点击与键盘输入调用同一组处理函数。程序支持四则运算、小数、退格、清除，并处理了连续小数点、连续运算符、除数为 0、结果后继续输入等边界情况。全部 23 项自动化测试通过，代码通过 Git 分阶段提交。下面按实验要求的条目逐项说明。

一、UI 界面布局与设计

主界面在 Qt 设计器中完成，保存为 calculator.ui。窗口用 QVBoxLayout 作为顶层垂直布局，自上而下依次是显示运算过程的 QLabel（对象名 expressionLabel）、显示数据与结果的 QLineEdit（对象名 display）以及一个 QGridLayout（对象名 gridLayout）。网格布局是 4 列 5 行，共 19 个 QPushButton：第一行是 C、⌫、%、÷，第二到第四行是数字 7~9、4~6、1~3 及乘号、减号、加号，最后一行是占两列宽的数字 0、小数点和等号。窗口缩放时由布局自动分配空间，不需要手工计算控件坐标；显示屏设为只读并右对齐，运算过程显示在它上方的小字标签里。

所有控件都显式命名，代码按对象名查找并连接，没有沿用 button、button_2 这类默认名称。主要控件与命名见表 1。

对象名	控件类型	作用
expressionLabel	QLabel	显示运算过程，如“12.5 ×”
display	QLineEdit	主显示屏，只读、右对齐
button0 ~ button9	QPushButton	数字 0 ~ 9
buttonDot	QPushButton	小数点
buttonAdd / buttonSub / buttonMul / buttonDiv	QPushButton	加、减、乘、除
buttonEquals	QPushButton	等于
buttonClear	QPushButton	清除（C）
buttonBackspace	QPushButton	退格（⌫）
buttonPercent	QPushButton	百分号（%）

表 1　主要控件与命名

关键代码（calculator.ui 中按键网格的片段）：

```
<layout class="QGridLayout" name="gridLayout">
  <item row="0" column="0"><widget class="QPushButton" name="buttonClear">...</widget></item>
  <item row="0" column="1"><widget class="QPushButton" name="buttonBackspace">...</widget></item>
  <item row="0" column="3"><widget class="QPushButton" name="buttonDiv">...</widget></item>
  <item row="4" column="0" colspan="2"><widget class="QPushButton" name="button0">...</widget></item>
```

```
</layout>
```

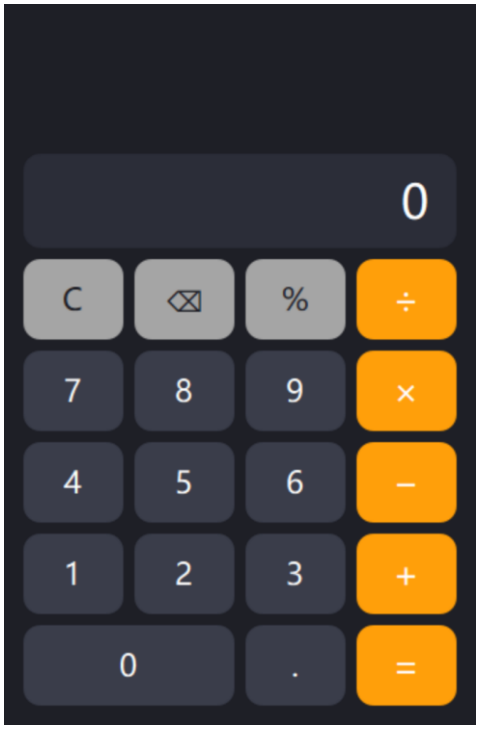


图 1　程序初始界面（Qt 设计器布局，鼠标与键盘共用同一套逻辑）

二、按键事件处理

数字、小数点、退格和清除四个操作分别由 enterDigit()、enterDot()、backspace() 和 clearAll() 完成，按钮的 clicked 信号和键盘事件都调用这几个函数，因此鼠标与键盘的行为完全一致。输入数字时，如果显示屏还是 0 就直接替换，避免出现“0123”这样的前导零；退格每次删除最后一位，删空后自动回到 0 并重新处于等待输入状态；清除则把第一个操作数、运算符和输入状态一起复位。

1. 避免同一操作数重复输入小数点

小数点的问题在于，如果允许第二次输入，显示屏会出现“1..”，之后用 toDouble() 解析就会失败，数值变成 0，后续运算全部出错。处理办法是在输入前检查显示屏里是否已经包含小数点，只要已经存在就直接返回，不再追加。

```
QString text = ui->display->text();
if (text.contains(QLatin1Char('.')))
    return;                         // 已经有一个小数点，忽略这次输入
if (text.isEmpty())
    text = QStringLiteral("0");
setDisplayText(text + QLatin1Char('.'));
```

另外，当显示屏刚被运算符或等号清空时按小数点，程序会自动补成“0.”，这样直接按小数

点输入小数也能正常工作。

2. 测试结果

输入 12.34 后按一次退格，显示屏由 12.34 变为 12.3（图 2）；直接按小数点得到 0.5；连续按三次小数点再加数字 2，最终显示 1.2，程序没有出现解析失败或显示异常。



图 2　退格（12.34 → 12.3）与小数输入

三、双操作数处理

计算器在任何时刻只需要记住“第一个操作数、待处理的运算符、第二个操作数是否已经开始输入”，因此用三个成员变量就能完整描述状态，见表 2。

成员变量	含义	变化时机
m_accumulator	第一个操作数，连续运算时保存中间结果	按下运算符、得到结果时更新
m_pendingOp	尚未参与计算的运算符（None 表示没有）	按下运算符时设置，计算完成后清空
m_freshOperand	下一次数字输入是否替换显示屏内容	按下运算符或等号后置为 true

表 2　计算状态变量

按下运算符时，如果此时还没有待处理运算符，就把当前显示的数值记为第一个操作数；如果已经有待处理运算符，说明用户输入了第二个操作数，先算出中间结果并回显，再把这个结果作为

新的第一个操作数，从而实现连续运算。计算失败（例如除数为 0）时不改变状态，只提示错误。

```
void Calculator::applyOperator(Op op)
{
    const double value = currentValue();
    if (m_pendingOp == Op::None) {
        m_accumulator = value;                  // 记下第一个操作数
    } else {
        double result = 0.0;
        if (!compute(m_accumulator, value, m_pendingOp, result)) {
            showError(QStringLiteral("除数不能为 0"));
            return;
        }
        m_accumulator = result;                 // 中间结果作为新的第一个操作数
        setDisplayText(formatNumber(result));
    }
    m_pendingOp = op;
    m_freshOperand = true;                      // 等待第二个操作数
}
```

按等号时调用 evaluate()：把第一个操作数、运算符和第二个操作数交给 compute() 计算，把结果显示出来并清空运算符；同时把 m_freshOperand 置为 true，这样接下来输入的数字会作为新的操作数，而不是接在结果后面。如果按等号时并没有待处理的运算符，就把当前显示直接作为结果。

四、连续操作与边界处理

连续运算的处理方式是：每按一次运算符就把上一次的中间结果保存到 m_accumulator，因此 1 + 2 + 3 + 4 = 会依次得到 3、6、10。在实现过程中一共发现并修复了 5 个边界问题，记录见表 3。

测试输入	原来的现象	原因分析	修改方法	修改后结果
1 再连按三次小数点 2	第一版允许继续追加小数点，显示屏出现 1..，按等号后解析失败变成 0	enterDot() 没有检查显示屏中是否已有小数点	输入前判断 text.contains('.')，已存在则直接返回	显示屏保持 1.2，输入过程正常（图 2）
5 + − × 3 =	第一版把“−”当成第二个操作数，用 0 参与计算，结果错误	applyOperator() 没有区分“刚按下运算符、还没输入第二个操作数”的情况	增加 m_freshOperand 判断，连续按运算符时只替换 m_pendingOp	按最后一个运算符计算，5 × 3 = 15
5 ÷ 0 =	第一版直接返回 inf，	compute() 没有判断	compute() 返回	显示“错误”和“除

	显示屏出现“inf”，之后运算全部失效	除数是否为 0	bool，除数为 0 时返回 false；界面显示“错误”并提示原因	数不能为 0”（图 3），输入数字即可重新计算
2 + 3 = 之后继续输入 4	第一版把 4 接在结果 5 后面，显示屏变成 54	得到结果后没有标记“下一个数字是新操作数”	evaluate() 结束后置 m_freshOperand = true，并在 enterDigit() 中重置累计值	输入 4 时显示屏直接变为 4，开始新的计算
输入 123 的过程中连按退格	第一版删空后显示屏变成空白	backspace() 没有处理删空的情况	删空或只剩负号时重置为 0，并回到等待新操作数状态	显示 0，随后可以继续输入或运算

表 3　异常与边界输入测试记录



图 3　除数为 0 的错误提示与连续运算 1+2+3+4=10

五、使用样式表改善界面

界面样式放在 style.qss 中，通过 resources.qrc 打进可执行文件，程序启动时用 setStyleSheet() 加载。样式表按功能给按钮分组：数字键使用深灰底配白字，运算符和等号使用橙色强调，C、⌫、% 使用浅灰底配深色字，显示屏使用更深的底色和更大的字号，并统一加了圆角。

```
QPushButton {
    background-color: #3a3d4a;  color: #ffffff;
```

```
    border: none;  border-radius: 12px;  font-size: 22px;
}
QPushButton#buttonAdd, QPushButton#buttonSub,
QPushButton#buttonMul, QPushButton#buttonDiv { background-color: #ff9f0a; font-size: 26px; }
QLineEdit#display {
    background-color: #2b2d38;  color: #ffffff;
    border-radius: 12px;  font-size: 34px;
}
```

主要改进有三点：一是用颜色区分数字键、运算符键和功能键，用户扫一眼就能分清按键类别；二是把显示屏字号调大并与按键拉开间距，读结果更清楚；三是给所有按钮统一圆角和悬停、按下状态，点击时有反馈。样式全部按对象名选择，没有改动任何逻辑代码。



图 4　样式表生效后的界面：结果显示与运算过程

六、键盘事件处理

键盘输入在 keyPressEvent() 中统一处理，按键与功能的对应关系见表 4。处理时先判断事件文本是不是单个数字（这样主键盘区和小键盘区的数字都能识别），其余按键再用 switch 按 Qt::Key 分派。

键盘按键	功能	调用的函数
0 ~ 9	输入数字	enterDigit()
.（句点或小键盘）	小数点	enterDot()
+ − *（或 x） /	加、减、乘、除	applyOperator()

Enter 或 =	等于	evaluate()
Backspace	退格	backspace()
Esc / Delete / C	清除	clearAll()
%	百分号	applyPercent()

表 4　键盘按键与功能对应表

鼠标与键盘复用同一套处理逻辑：按钮的 clicked 信号和键盘分支都只调用上面那几个函数，没有为键盘单独写一份运算代码，所以两种输入的结果必然一致。为了让键盘事件能到达主窗口，所有按钮和显示屏都设置为不接收焦点，这样即使用鼠标点过按钮，焦点仍留在窗口上，键盘输入不会失效。

```
void Calculator::keyPressEvent(QKeyEvent *event)
{
    const QString text = event->text();
    if (text.size() == 1 && text.at(0).isDigit()) {   // 数字
        enterDigit(text);
        event->accept();
        return;
    }
    switch (event->key()) {
    case Qt::Key_Period:    enterDot();               break;   // .
    case Qt::Key_Plus:      applyOperator(Op::Add);   break;   // +
    case Qt::Key_Asterisk:  applyOperator(Op::Multiply); break;// *
    case Qt::Key_Slash:     applyOperator(Op::Divide); break;  // /
    case Qt::Key_Return:    evaluate();               break;   // Enter
    case Qt::Key_Backspace: backspace();              break;   // 退格
    case Qt::Key_Escape:    clearAll();               break;   // 清除
    default: QWidget::keyPressEvent(event); return;
    }
    event->accept();
}
```

测试时分别用键盘输入 6 * 7 = 和鼠标点击 6、×、7、=，两种方式得到的都是 42（图 5）；把两者混用（鼠标点击数字、键盘按运算符、再鼠标点击数字）同样能得到正确结果，说明鼠标与键盘确实走的是同一套逻辑。



图 5　键盘输入与鼠标点击得到相同结果 6 × 7 = 42

七、Git 提交日志

代码用 Git 分阶段提交，完整地体现了“搭框架 → 做界面 → 写功能 → 修问题 → 补测试 → 写文档”的过程，提交记录见表 5，提交历史截图见图 6。

序号	提交说明	主要内容
1	chore: 初始化 Qt 计算器项目骨架	qmake 与 CMake 工程、入口文件、.gitignore
2	feat(ui): 使用 UI 设计器完成界面布局与控件命名	calculator.ui
3	feat: 添加界面样式表与资源文件	style.qss、resources.qrc
4	feat: 实现四则运算、键盘输入与基本操作	calculator.h / calculator.cpp 第一版
5	fix: 处理连续小数点、连续运算符与除零等异常输入	边界处理与错误提示
6	test: 添加计算器行为自动化测试（23 个用例）	tests/ 目录
7	docs: 添加运行截图生成工具与实验运行截图	报告插图
8	fix(ui): 补充退格按钮，移除冗余的正负号按钮	修复界面缺少退格键的问题
9	docs: 更新运行截图与演示视频（含退格按钮）	重新生成素材
10	docs: 添加报告生成脚本与实验报告模板	tools/build_report.py 与报告模板

表 5　Git 提交记录



图 6　Git 提交历史

本报告与演示视频作为第 11 次提交（docs: 添加实验报告与演示视频）一并提交。

八、AI 辅助开发记录

下面记录一次比较有代表性的 AI 辅助开发过程：除数为 0 时的错误处理。

1. 遇到的问题

第一版程序没有判断除数是否为 0，按 5 ÷ 0 = 时显示屏直接出现“inf”，而且因为内部状态已经变成无效值，之后继续输入数字也算不对。

2. 使用的提示词

“我用 Qt 写了一个计算器，做除法时按 5 ÷ 0 = 显示屏出现 inf，而且之后继续输入也不正常。请问怎么判断除数是不是 0，并让程序在给出错误提示之后还能继续使用？”

3. AI 给出的方案

AI 建议在计算函数里判断除数是否为 0，如果为 0 就返回失败而不要继续做除法；同时建议用一个成员变量标记错误状态，出错后先把状态复位再响应新的输入；还提醒比较浮点数不要用 == 0，而应该用 Qt::qFuzzyIsNull()。

4. 实际运行结果

按建议修改后，按 5 ÷ 0 = 会显示错误提示，之后输入 7 并按 + 3 = 能正常得到 10，

qFuzzyIsNull() 的用法也是正确的，这一部分直接用在了最终代码里。

5. 存在的问题

AI 的第一版方案把整句提示“除数不能为 0”直接写进显示屏，而显示屏字号是 34 像素，这句话在窗口里显示不全；另外它建议在每一个输入函数开头都重复写一遍“如果出错就复位”，代码比较啰嗦。

6. 我做的修改

把长提示拆开：显示屏只显示“错误”两个字，完整原因显示在上方操作过程的小字标签里，既看清了提示又不挤占显示屏；复位逻辑只保留在输入数字的入口处统一处理，去掉了重复判断；最后补了自动化测试，用 5 ÷ 0 = 显示错误、再输入 7 + 3 = 得到 10 这两个断言确认修改有效。

九、测试结果

程序另写了 tests/test_calculator.cpp，通过发送真实的键盘事件和按钮点击事件驱动界面，再读取显示屏文本来判断结果，共 11 组、23 项断言，全部通过。用例与结果见表 6。

用例	测试内容	期望结果	实际结果
1	12 + 34、7 − 10、6 × 7、8 ÷ 2	46、-3、42、4	一致
2	1.5 + 2.25、0.1 + 0.2	3.75、0.3	一致（未出现浮点误差）
3	连按小数点 1...2、直接按小数点	1.2、0.5	一致
4	5 + − × 3 =	15	一致
5	123 退格、退到空、退格后重新计算	12、0、10	一致
6	1 + 2 + 3 + 4 =	10	一致
7	5 ÷ 0 =、出错后输入 7、7 + 3 =	错误、7、10	一致
8	2 + 3 = 后输入 40、40 + 2 =	5、40、42	一致
9	2 + 3 = 之后按 × 4 =	20	一致
10	鼠标 12.5 × 4 =、鼠标与键盘混合 9 + 3 =	50、12	一致
11	12.34 退格成 12.3 后加 1	13.3	一致

表 6　自动化测试用例与结果（23 项断言全部通过）

按实验要求还录制了 2～3 分钟的演示视频 docs/演示视频.mp4，依次演示了鼠标完成带小数的运算（12.5 × 4 = 50）、键盘计算（1.5 + 2.25 = 3.75）、退格（键盘与鼠标各演示一次）、清除后重新计算、除数为 0 的处理、连续运算（1 + 2 + 3 + 4 = 10）以及两种异常输入（连续

小数点、连续运算符），视频结尾展示了本项目的 Git 提交历史。

十、实验总结

1. 出错分析与解决过程

最花时间的问题不是运算本身，而是两处与状态相关的错误。第一处是键盘输入时有时完全没反应，原因是按钮默认会抢占焦点，用鼠标点过按钮之后焦点落在按钮上，键盘事件就不再送到主窗口；把显示屏和所有按钮的焦点策略设为 NoFocus、让窗口自己持有焦点之后问题就消失了。第二处是连续按运算符时程序会用错误的数据参与计算，后来发现是缺少“是否在等待第二个操作数”这个状态，补上 m_freshOperand 判断后解决。这两个问题让我意识到，界面程序里“状态”比“公式”更容易出错。

2. 本次实验小结

通过这次实验我熟悉了 Qt 设计器搭界面、信号与槽的连接方式、QVBoxLayout 与 QGridLayout 的配合使用、键盘事件的重写，以及用样式表美化界面。同时也体会到把运算逻辑集中到少数几个函数里的好处：鼠标和键盘只要调用同一组函数，行为就自然一致，出现问题时也只需要改一个地方。另外，用自动化测试代替手工反复点击之后，每次修改都能很快确认没有把原来的功能改坏。

3. AI 辅助开发的体会

AI 在这个实验里帮了不少忙：它能很快给出判断除数为 0 的写法、提醒浮点数比较要用 qFuzzyIsNull()，也能解释 Qt 布局的使用方式。但它给出的代码不能直接照抄使用，比如把整句错误提示塞进显示屏，在实际界面里会显示不全；它的方案里也有重复的复位判断。我的做法是先把它给的思路跑起来，看真实运行结果，再按界面尺寸和代码结构去调整，最后用测试用例确认修改确实生效。总的说来，AI 负责提供思路和初稿，验证、调整和取舍还是得自己做。